

PKU–Zurich PhD Summer School on Machine Learning for Macroeconomics and Finance

From representative-agent Brock–Mirman through stochastic models, KKT
constraints, and loss kernels

Simon Scheidegger

HEC Lausanne · University of Zurich

July 6–10, 2026 · Beijing, China

Based on: Azinovic, Gaegauf & Scheidegger (2022), *International Economic Review* 63(4), 1471–1525.

https://liboecon.com/2026_summer_school.html

Roadmap of This Lecture

Motivation

- ▶ Heterogeneity and high dimensions
- ▶ Nonlinearities and kinks
- ▶ Irregular ergodic sets
- ▶ Our solution: DEQNs

From ANNs to DEQNs

- ▶ DNNs as function approximators
- ▶ Supervised vs. self-supervised loss
- ▶ The DEQN training algorithm

Brock–Mirman Benchmark

- ▶ Analytical test case
- ▶ DEQN implementation

Parallelization & HPC

- ▶ Data parallelism (MPI/Horovod)
- ▶ Scalability to many nodes

Wrap-up + Hands-on

- ▶ Four notebooks: BM (01_01), BM+shocks (01_02), exercises blank/solved (01_03/01_04)
- ▶ Exercise preview: KKT + Fischer–Burmeister, 6-period OLG

Learning objectives

By the end of this lecture you should be able to:

- ▶ Formulate dynamic stochastic economic models as equilibrium systems and identify computational challenges in high dimensions
- ▶ Replace grid-based collocation with self-supervised neural-network training using equilibrium errors as the loss function
- ▶ Build and train Deep Equilibrium Nets (DEQNs) to solve representative-agent and heterogeneous-agent models
- ▶ Implement simulated paths from the model's ergodic distribution to construct training data without explicit grids
- ▶ Parallelize DEQN training across multiple nodes and scale solutions to 100+ dimensions

Part I

Motivation

Heterogeneity Matters

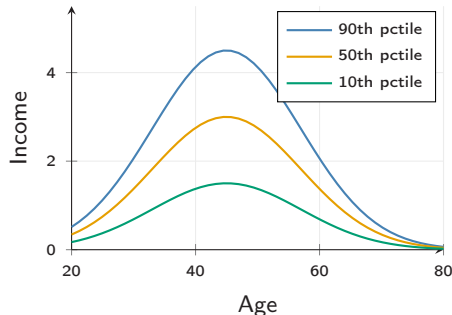
Key questions: inequality over the life cycle; idiosyncratic vs. aggregate risk; distributional effects of policy.

Quantitative punchline:

- ▶ **Hand-to-mouth** HH: $MPC \approx 1$; unconstrained: $MPC \approx 0.05$
- ▶ Representative agent averages these out, misses the policy response

Kaplan, Moll & Violante 2018; Bilbiie 2008; Krueger et al. 2016

⇒ Heterogeneity is one of several forces that **blow up** d (also: many countries, many assets, climate state, ...).



Stylized income-by-age profiles

Dynamic Stochastic General Equilibrium Models

Generic formulation: find policy functions $p(\cdot)$ such that

$$\mathbb{E}\left[G(\mathbf{x}_t, \mathbf{x}_{t+1}, p(\mathbf{x}_t), p(\mathbf{x}_{t+1})) \mid \mathbf{x}_t, p(\mathbf{x}_t)\right] = 0$$

with transition law $\mathbf{x}_{t+1} = h(\mathbf{x}_t, p(\mathbf{x}_t), \varepsilon_{t+1})$.

Four computational challenges (Azinovic, Gaegauf & Scheidegger, 2022):

1. **Stochasticity**: nontrivial expectations over ε_{t+1} and the ergodic distribution
2. **High-dimensional** state space: heterogeneity, many assets, many shocks all push d up
3. **Strong nonlinearities / kinks** from occasionally binding constraints and regime switches
4. **Irregular geometry** of the ergodic set: recurrent states are not a hypercube

No classical method addresses all four jointly: sparse grids cover (1–3) but become impractical past $d \sim 20$ under conventional smoothness assumptions; Gaussian processes cover (1,2,4) but their $O(N^3)$ training cost limits them to small N .

What is High-Dimensional?

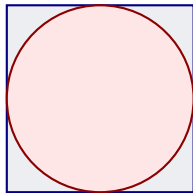
Dimensions d	Grid points n^d	Runtime
1	10	10 seconds
2	100	2 minutes
5	10^5	1 day
10	10^{10}	317 years
20	10^{20}	~ 3 trillion years

Assumes $n = 10$ grid points per dimension and 1 second per evaluation.

Three remedies:

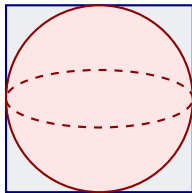
1. **Sparse grids** (Brumm & Scheidegger, 2017)
2. **Random grids** (Rust, 1997)
3. **Neural networks** (this lecture)

Volumes in High Dimensions



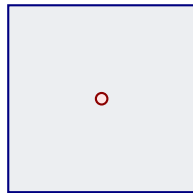
2D

$$V_{\text{sphere}}/V_{\text{cube}} = \pi/4 \approx 0.785$$



3D

$$V_{\text{sphere}}/V_{\text{cube}} = \pi/6 \approx 0.524$$



100D

$$V_{\text{sphere}}/V_{\text{cube}} \approx 1.87 \times 10^{-70}$$

Implication

In high dimensions, **almost all volume is in the corners**. Grid-based methods waste most of their evaluation points in regions that barely contribute to the integral or approximation.

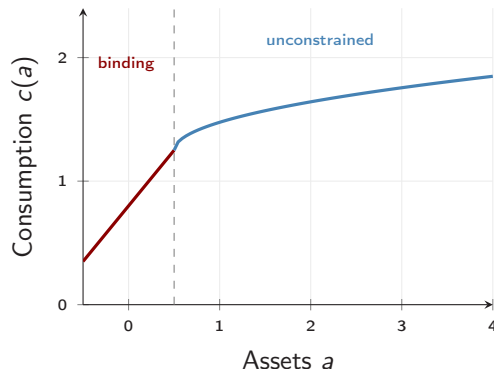
Nonlinearities and Kinks

Where do kinks come from?

- ▶ **Borrowing constraints:** $a_{t+1} \geq \underline{a}$
- ▶ **Zero lower bound:** $i_t \geq 0$
- ▶ **Short-sale / participation:** corner solutions
- ▶ **Default, regime switches, indivisibilities**

Occasionally binding constraints $\Rightarrow$ policy functions are **non-smooth** at the boundary of the binding region.

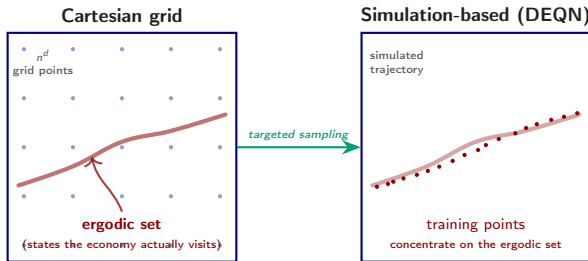
Polynomial / spectral methods lose accuracy; sparse grids can resolve kinks but only for $d \lesssim 20$. Neural networks handle kinks natively via ReLU-type activations.



Kinked consumption policy at the borrowing limit

The Ergodic Set Is Not a Hypercube

Grid-based methods tile the full hypercube $\mathbf{x} \in [\mathbf{x}_{\min}, \mathbf{x}_{\max}]^d$. **But** simulated states drawn from the model's law of motion fill only a **thin, irregular** subset, the **ergodic set** (states actually visited by the economy), often $\ll 1\%$ of the hypercube (Judd, Maliar & Maliar, 2011).



A full Cartesian grid **wastes** most of its work in regions the economy never visits $\Rightarrow$ motivates **simulation-based, grid-free** methods.

Abstract Problem Formulation

We need a numerical method that satisfies:

1. **Global approximation:** avoid purely local perturbation solutions
2. **Approximate solutions:** accept small residual error $\varepsilon > 0$
3. **Grid-free in practice:** avoid explicit tensor-product growth n^d
4. **Flexible geometry:** handle nonlinearities, kinks, and irregular ergodic sets
5. **Parallel-friendly:** benefit from **parallelization** on modern hardware

⇒ DEQNs address this wishlist well in practice, while **mitigating** rather than eliminating high-dimensional costs.

Our Solution: Deep Equilibrium Nets

Core idea (Azinovic, Gaegauf & Scheidegger, 2022):

Replace the traditional grid-based solution with a **neural network** trained to satisfy economic equilibrium conditions.

Three Key Ingredients

1. **Equilibrium error as loss function:** use the economic equations $G(\cdot) = 0$ directly, with **no labeled data** needed
2. **Stochastic gradient descent:** optimize network parameters θ via SGD on mini-batches
3. **Simulated paths:** draw training points from the **ergodic distribution** of the model itself

⇒ This turns **solving** an economic model into **training** a neural network.

It avoids explicit state-space grids, but expectation accuracy, sampling quality, and optimization still matter.

Part II

From ANNs to Deep Equilibrium Nets

What is a Deep Neural Network?

A deep neural network (DNN) $\mathcal{N}_\theta : \mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^{d_{\text{out}}}$ is a **parametric function approximator**.

Universal approximation (Cybenko 1989, Hornik et al. 1989):

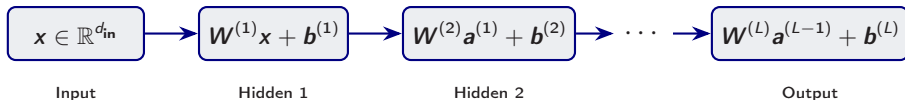
$$\forall f \in C(\mathcal{K}, \mathbb{R}), \forall \varepsilon > 0, \exists \theta^* : \sup_{\mathbf{x} \in \mathcal{K}} \|\mathcal{N}_{\theta^*}(\mathbf{x}) - f(\mathbf{x})\| < \varepsilon$$

($\mathcal{K} \subset \mathbb{R}^{d_{\text{in}}}$ compact; the norm is the supremum over $\mathcal{K}$.)

In economics:

- ▶ **Input:** state variables $\mathbf{x}_t \in \mathbb{R}^d$ (capital, productivity, wealth distribution, ...)
- ▶ **Output:** policy variables (consumption, savings, prices, ...)
- ▶ **Parameters** θ : weights and biases, found by **training**

DNN Layer Structure



Each layer performs an **affine transformation**:

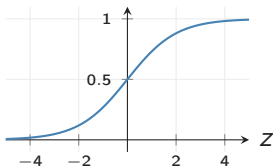
$$\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad \ell = 1, \dots, L$$

where $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell} \times n_{\ell-1}}$ and $\mathbf{b}^{(\ell)} \in \mathbb{R}^{n_{\ell}}$.

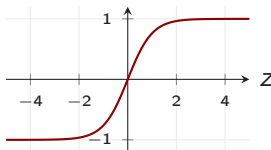
Without nonlinear activations, the entire network collapses to a single affine map.

Activation Functions

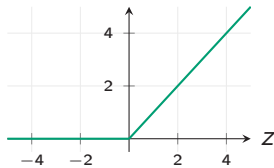
Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$



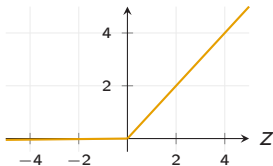
Tanh: $\tanh(z)$



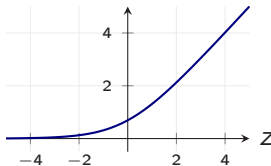
ReLU: $\max(0, z)$



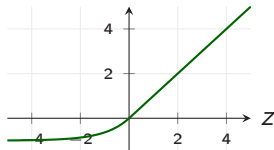
Leaky ReLU: $\max(0.01z, z)$



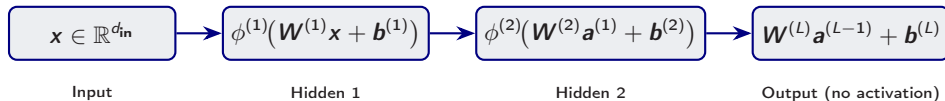
Softplus: $\ln(1 + e^z)$



ELU: z if $z \geq 0$; $e^z - 1$ if $z < 0$



DNN with Nonlinear Activations



Each hidden layer applies a **nonlinear activation** $\phi^{(\ell)}$:

$$\mathbf{a}^{(\ell)} = \phi^{(\ell)}(\mathbf{W}^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}), \quad \ell = 1, \dots, L-1$$

- ▶ The output layer is typically **linear** (for regression) or uses **softplus** (to enforce positivity, e.g. consumption)
- ▶ The full network: $\mathcal{N}_{\theta}(\mathbf{x}) = \mathbf{W}^{(L)}\mathbf{a}^{(L-1)} + \mathbf{b}^{(L)}$

Finding Parameters: Supervised Learning

Standard approach (when labeled data $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ are available):

1. **Collect labeled data:** $\mathbf{x}_i \xrightarrow{\text{true mapping}} y_i$
2. **Define a loss function:**

$$\ell_{\theta}^{\text{sup}} = \frac{1}{N} \sum_{i=1}^N \|y_i - \mathcal{N}_{\theta}(\mathbf{x}_i)\|^2 \quad (\text{mean squared error})$$

3. **Update via SGD** (η = learning rate; the symbol α is reserved for the Cobb–Douglas capital share later in the deck):

$$\theta^{\text{new}} = \theta^{\text{old}} - \eta \cdot \left. \frac{\partial \ell_{\theta}}{\partial \theta} \right|_{\theta=\theta^{\text{old}}}$$

This is the recipe from Day 1's machine-learning introduction. But in economics, **we rarely have labeled data**...

The “Data Problem” in Economics

Deep learning excels when massive labeled datasets are available.

But in computational economics:

- ▶ We solve **functional equations**, not classification/regression tasks
- ▶ There are **no labeled pairs** $(\mathbf{x}_i, y_i)$
- ▶ Traditional approach: **time iteration on a grid**, but grids do not scale

Key insight:

- ▶ We **know** the structural equations $G(\cdot) = 0$
- ▶ We can **check** how well a candidate policy satisfies them
- ▶ $\Rightarrow$ Use the **equilibrium error** as the loss function!

Traditional Approach: Time Iteration on a Grid

Algorithm: Grid-Based Collocation

Input: Grid $\mathcal{G} = \{\mathbf{x}_1, \dots, \mathbf{x}_M\} \subset \mathbb{R}^d$, initial guess $p^{(0)}$
for $k = 0, 1, 2, \dots$ until convergence **do**
 for each grid point $\mathbf{x}_i \in \mathcal{G}$ **do**
 Solve $G(\mathbf{x}_i, p^{(k)}(\mathbf{x}_i), p^{(k)}) = 0$ for $p^{(k+1)}(\mathbf{x}_i)$
 end for
 Interpolate $p^{(k+1)}$ between grid points
end for

Problems:

- ▶ **Cartesian** grids need $M = n^d$ points, infeasible beyond $d \sim 4-5$ at sensible resolution (10^5 at $d=5$, 10^{10} at $d=10$)
- ▶ **Sparse grids** (Smolyak, adaptive) extend the practical reach only to $d \sim 20-50$
- ▶ Heterogeneous-agent or many-country DSGE models routinely need $d \gg 50$, beyond any grid-based method

The DEQN Loss Function

Key idea: replace labeled data with **economic equilibrium conditions**.

Let $G(\mathbf{x}, f(\mathbf{x})) = 0$ be the system of equilibrium equations. Define:

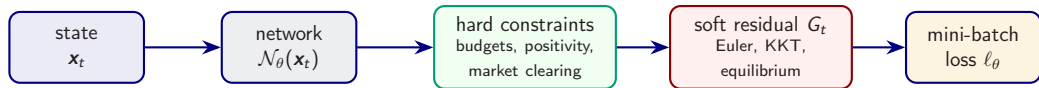
$$\ell_{\theta} = \frac{1}{N} \sum_{i=1}^N \|G(\mathbf{x}_i, \mathcal{N}_{\theta}(\mathbf{x}_i))\|^2$$

- ▶ $\mathcal{N}_{\theta}(\mathbf{x}_i)$ is the neural net's **predicted policy** at state $\mathbf{x}_i$
- ▶ $G(\cdot)$ measures the **equilibrium error** (e.g. Euler equation residual)
- ▶ Small ℓ_{θ} means the policy nearly satisfies the equations on the sampled states

$\Rightarrow$ **No labels needed**; we call this **self-supervised** learning (sometimes labelled “unsupervised” in the narrow no-labels sense).

In stochastic models, expectations inside G are approximated, e.g. by path averaging or quadrature.

Inside One DEQN Training Step



- ▶ Encode what can be enforced **exactly** in the architecture.
- ▶ Minimize only the genuinely non-closed-form equilibrium conditions.
- ▶ Draw states from the model's own simulation, so training focuses on the ergodic set.

Training Algorithm for DEQNs

Algorithm 1: Deep Equilibrium Net Training

Input: Initial state $\mathbf{x}_0$, network $\mathcal{N}_\theta$, learning rate η , episodes E , simulation horizon T_{sim} , training steps T_{train}

for episode $e = 1, \dots, E$ **do**

Simulate path: $\mathbf{x}_0 \rightarrow \mathbf{x}_1 \rightarrow \dots \rightarrow \mathbf{x}_{T_{\text{sim}}}$ using $\mathcal{N}_\theta$ and transition law

for gradient step $t = 1, \dots, T_{\text{train}}$ **do**

 Draw mini-batch $\mathcal{B} \subset \{\mathbf{x}_0, \dots, \mathbf{x}_{T_{\text{sim}}}\}$

 Compute loss: $\ell_\theta = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x}_i \in \mathcal{B}} \|G(\mathbf{x}_i, \mathcal{N}_\theta(\mathbf{x}_i))\|^2$

 Update: $\theta \leftarrow \theta - \eta \cdot \nabla_\theta \ell_\theta$

end for

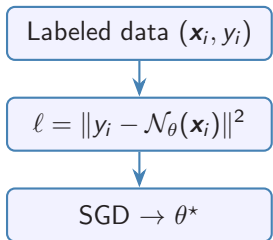
end for

Output: Trained network $\mathcal{N}_{\theta^*}$ approximating the policy function

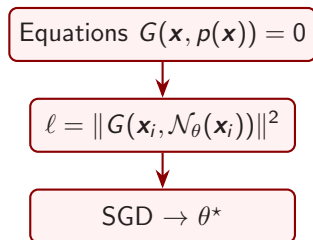
Note: Training points come from the **model's own simulation**, so the network learns on the ergodic distribution, focusing on economically relevant regions.

Supervised vs. Self-Supervised (DEQN) Learning

Supervised



DEQN (Self-Supervised)



Key difference: DEQNs replace labeled data with structural economic equations.

DEQN: Key Recap

Replace Labels

Instead of labeled data $(\mathbf{x}_i, y_i)$, use the economic equilibrium equations $G(\cdot) = 0$ as the loss.

Replace Grids

Instead of n^d grid points, use a neural network $\mathcal{N}_\theta$ that maps states to policies **globally**.

Replace TI

Instead of time iteration with interpolation, use SGD on simulated paths from the model.

Result: A grid-free, simulation-based method that can handle much higher-dimensional models than standard grids and maps naturally to GPUs.

Part III

Brock–Mirman Benchmark

Brock–Mirman (1972): Setup

Social planner's problem. Choose a consumption sequence to maximize expected lifetime log utility:

$$\max_{\{C_t\}_{t=0}^{\infty}} \mathbb{E} \left[\sum_{t=0}^{\infty} \beta^t \ln C_t \right]$$

Resource constraint (full depreciation, $\delta = 1$):

$$C_t + K_{t+1} = z_t K_t^{\alpha}, \quad K_0 > 0 \text{ given.}$$

Stochastic productivity. TFP z_t follows an AR(1) in logs:

$$\ln z_{t+1} = \varrho \ln z_t + \sigma_z \epsilon_{t+1}, \quad \epsilon_{t+1} \sim \mathcal{N}(0, 1).$$

State & control. State $x_t = (K_t, z_t)$; control C_t . Capital then follows from the budget as $K_{t+1} = z_t K_t^{\alpha} - C_t$.

Next two slides: derive the Euler equation from first principles, then obtain the closed-form policy – the benchmark we will later compare the DEQN solution to.

Deriving the Euler Equation (Lagrangian)

Step 1. Lagrangian. Attach a multiplier $\lambda_t \geq 0$ to each period's budget constraint:

$$\mathcal{L} = \mathbb{E} \left[\sum_{t=0}^{\infty} \beta^t \left[\ln C_t - \lambda_t (C_t + K_{t+1} - z_t K_t^\alpha) \right] \right].$$

Step 2. Take partial derivatives with respect to the choices at date t :

Consumption FOC $\partial \mathcal{L} / \partial C_t = 0$:

$$\beta^t \left(\frac{1}{C_t} - \lambda_t \right) = 0 \implies \lambda_t = \frac{1}{C_t}.$$

Capital FOC $\partial \mathcal{L} / \partial K_{t+1} = 0$:

$$-\beta^t \lambda_t + \beta^{t+1} \mathbb{E} [\lambda_{t+1} \alpha z_{t+1} K_{t+1}^{\alpha-1}] = 0.$$

Step 3. Eliminate λ . Substitute $\lambda_t = 1/C_t$, $\lambda_{t+1} = 1/C_{t+1}$ and divide by β^t :

$$\boxed{\frac{1}{C_t} = \beta \mathbb{E} \left[\frac{\alpha z_{t+1} K_{t+1}^{\alpha-1}}{C_{t+1}} \right]} \quad (\text{stochastic Euler equation})$$

Reading: marginal utility of today's consumption = discounted expected marginal utility of tomorrow's consumption $\times$ gross marginal product of capital at $t+1$. The transversality condition $\lim_{t \rightarrow \infty} \mathbb{E} [\beta^t \lambda_t K_{t+1}] = 0$ rules out bubbles.

Closed-Form Policy via Guess-and-Verify

Guess. Consumption is a constant fraction $1 - \phi$ of output:

$$C_t = (1 - \phi) z_t K_t^\alpha, \quad \phi \in (0, 1) \text{ to be determined.}$$

The budget then implies $K_{t+1} = \phi z_t K_t^\alpha$.

Substitute into the Euler equation. With $C_{t+1} = (1 - \phi) z_{t+1} K_{t+1}^\alpha$,

$$\frac{\alpha z_{t+1} K_{t+1}^{\alpha-1}}{C_{t+1}} = \frac{\alpha z_{t+1} K_{t+1}^{\alpha-1}}{(1 - \phi) z_{t+1} K_{t+1}^\alpha} = \frac{\alpha}{(1 - \phi) K_{t+1}}.$$

Note: z_{t+1} cancels, so the conditional expectation collapses (log + Cobb–Douglas + full depreciation).

Solve for ϕ . Plug $K_{t+1} = \phi z_t K_t^\alpha$ on both sides of the Euler:

$$\underbrace{\frac{1}{(1 - \phi) z_t K_t^\alpha}}_{\text{LHS}} = \beta \cdot \underbrace{\frac{\alpha}{(1 - \phi) \phi z_t K_t^\alpha}}_{\text{RHS}} \implies 1 = \frac{\beta \alpha}{\phi} \implies \phi = \beta \alpha.$$

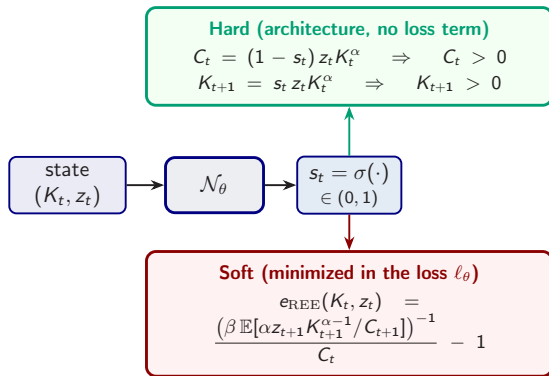
Closed-form policy.

$$C_t = (1 - \beta \alpha) z_t K_t^\alpha, \quad K_{t+1} = \beta \alpha z_t K_t^\alpha$$

This benchmark provides an analytic target against which we will measure the DEQN's trained policy.

Brock–Mirman: Hard vs. Soft Constraints

Split the equilibrium conditions by what the architecture can enforce exactly.



Why bother? A softplus head on C_t alone would secure $C_t > 0$ but could push $K_{t+1} < 0$. The sigmoid-savings head removes *both* failure modes before training starts. Only the genuinely non-closed-form condition, the Euler equation, enters the loss.

Brock–Mirman: Residual, Expectation, Loss

Pathwise residual used in the notebook (full-depreciation $\delta = 1$ benchmark):

$$G_t = 1 - \beta \frac{C_t}{C_{t+1}} \cdot \alpha z_{t+1} K_{t+1}^{\alpha-1},$$

with

$$K_{t+1} = s_t z_t K_t^\alpha, \quad C_{t+1} = (1 - s_{t+1}) z_{t+1} K_{t+1}^\alpha.$$

Partial depreciation. The uncertainty notebook uses $\delta = 0.1$; the marginal productivity of capital then becomes $\alpha z_{t+1} K_{t+1}^{\alpha-1} + (1 - \delta)$ and $K_{t+2} = z_{t+1} K_{t+1}^\alpha + (1 - \delta) K_{t+1} - C_{t+1}$. The closed-form $\phi = \alpha\beta$ only holds at $\delta = 1$.

Training loss:

$$\ell_\theta = \frac{1}{N} \sum_{i=1}^N |G_t^{(i)}|^2.$$

- ▶ Simulate states (K_t, z_t) forward
- ▶ At each state, take $\mathbb{E}_t[\cdot]$ over z_{t+1} by 5-node Gauss–Hermite quadrature

Expectation Handling

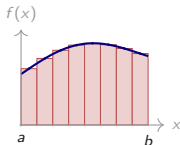
Brock–Mirman notebook: Gauss–Hermite quadrature (5 nodes) for the conditional expectation at each state.

Richer models: same idea with more nodes, or product / sparse grids in higher dimensions, see next three slides.

Numerical Integration: Why and How

Why an integral always shows up. Euler equations, Bellman operators, REE diagnostics, climate damages, structural moments, all reduce to evaluating $\mathbb{E}[g(\varepsilon)] = \int g(\varepsilon) \mu(\varepsilon) d\varepsilon$. Closed forms are rare, so we approximate by $\sum_q w_q g(\varepsilon_q)$. Two paradigms underlie every rule.

Deterministic: tile the area, integral $\rightarrow$ sum

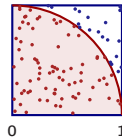


With $\Delta x = (b - a)/N$, $x_i = a + i\Delta x$, $I = \int_a^b f(x) dx \approx \sum_i w_i f(x_i)$:

- ▶ **Midpoint:** $I_N^{\text{mid}} = \Delta x \sum_{i=1}^N f(x_i^{\text{mid}})$, $\mathcal{O}(N^{-2})$
- ▶ **Trapezoid:** $I_N^{\text{trap}} = \frac{\Delta x}{2} [f(x_0) + 2 \sum_{i=1}^{N-1} f(x_i) + f(x_N)]$, $\mathcal{O}(N^{-2})$
- ▶ **Simpson:** parabolic arcs through triples, $\mathcal{O}(N^{-4})$
- ▶ **Gauss-Hermite:** optimal nodes/weights for e^{-x^2} , $\mathcal{O}(N^{-(2Q-1)})$

d -dim Cartesian grid: cost $\mathcal{O}(N^d)$, **curse of dimensionality**.

Random: throw darts to estimate π



Throw N uniform darts at $\Omega = [0, 1]^2$:

$$\frac{\pi}{4} = \int_0^1 \int_0^1 1_{[x^2 + y^2 \leq 1]} dx dy \approx \frac{N_{\text{in}}}{N}$$

$$\hat{\pi}_N = 4N_{\text{in}}/N, \quad \text{error } \mathcal{O}(N^{-1/2})$$

Independent of dimension, but slow.

Quadrature, I: Gauss–Hermite

Setup. Generic shock dimension d , with $\varepsilon' \sim \mathcal{N}(0, I_d)$. Brock–Mirman has $d = 1$ (one productivity shock); the IRBC session (Session 3) will use $d = N + 1$ (N country shocks plus one aggregate). We develop the framework here so it is ready for both cases.

One dimension. Gauss–Hermite uses Q nodes $\{\varepsilon_q\}$ (Hermite-polynomial roots) and weights $\{w_q\}$:

$$\mathbb{E}[h(\varepsilon)] = \int_{-\infty}^{\infty} h(\varepsilon) \frac{e^{-\varepsilon^2/2}}{\sqrt{2\pi}} d\varepsilon \approx \sum_{q=1}^Q w_q h(\varepsilon_q),$$

exact for univariate polynomials of degree $\leq 2Q - 1$.

Tensor product in d dimensions. Combine 1D rules along each axis:

$$\mathbb{E}[h(\varepsilon')] \approx \sum_{q_1=1}^Q \cdots \sum_{q_d=1}^Q \left(\prod_j w_{q_j} \right) h(\varepsilon_{q_1}, \dots, \varepsilon_{q_d}).$$

- **Strength:** polynomial-exact in each coordinate, very accurate at low d . For $d = 1$ (Brock–Mirman) a $Q = 3$ rule is essentially exact.
- **Weakness:** cost Q^d blows up exponentially. For $Q = 3$, $d = 11$ (IRBC, $N = 10$): $\sim 1.8 \times 10^5$ nodes per state.

Quadrature, II: Monomial (Stroud-3) Rule

Idea. Drop full polynomial-exactness in every coordinate, keep exactness for total degree ≤ 3 . Place nodes only on the principal axes of the d -dimensional shock space:

$$\mathbb{E}[h(\boldsymbol{\varepsilon}')] \approx \frac{1}{2d} \sum_{k=1}^d \left[h(+\sqrt{d} \mathbf{e}_k) + h(-\sqrt{d} \mathbf{e}_k) \right]$$

where $\mathbf{e}_k$ is the k -th standard basis vector, giving $2d$ nodes at common radius $\sqrt{d}$. For Brock–Mirman ($d = 1$) this reduces to two nodes at ± 1 (exact for univariate polynomials of degree ≤ 3); for IRBC ($d = N + 1$) it gives $2(N + 1)$ nodes.

What it integrates exactly

- ▶ $\mathbb{E}[\varepsilon'_i] = \mathbb{E}[\varepsilon_i'^3] = 0$ ($\pm$ -symmetry).
- ▶ $\mathbb{E}[\varepsilon'_i \varepsilon'_j] = 0$, $i \neq j$ (each node lies on one axis).
- ▶ $\mathbb{E}[\varepsilon_i'^2] = 1$ (variance preserved).

Where it stops

- ▶ $\mathbb{E}[\varepsilon_i'^4]$ gives d vs. truth 3, a controlled fourth-order bias.

Why DEQNs love it

The integrand $h(\boldsymbol{\varepsilon}')$ is the policy network composed with model primitives, smooth and only mildly nonlinear. Degree-3 exactness is enough for Euler-error tolerances of $\sim 10^{-3}$ on IRBC-class problems.

(Pichler 2011; Maliar & Maliar 2014)

Quadrature, III: Cost and Rules of Thumb

Per-state quadrature cost ($Q = 3$ for the tensor-product rule):

N	shocks $N+1$	tensor product 3^{N+1}	monomial $2(N+1)$	speed-up
2	3	27	6	$\sim 5\times$
5	6	729	12	$\sim 60\times$
10	11	177,147	22	$\sim 8,000\times$
20	21	$\sim 10^{10}$	42	$\sim 2 \times 10^8\times$

Default rules of thumb

- ▶ $N+1 \leq 4$: tensor-product GH, $Q=5$.
- ▶ $5 \leq N+1 \lesssim 30$: Stroud-3 monomial.
- ▶ $N+1 \gtrsim 30$ or fat-tailed integrands: QMC (Sobol + inverse-CDF).

Practical notes

- ▶ Only the quadrature **kernel** changes; same DEQN loss, same training loop.
- ▶ Validate by tightening the rule and checking the Euler-error histogram is stable.
- ▶ Full derivations: lecture script §3.5.

Diagnostics: How Do We Know Training Worked?

Never trust a loss curve alone. On an independent *test* simulation of the trained policy, run these four complementary checks:

1. **Euler-error distribution.** Histogram $\log_{10} |e_{\text{REE}}|$ on train and test states; both should peak near 10^{-3} or below, with no fatter right tail on test.
2. **Policy-function benchmark.** Overlay the DEQN policy on the analytic $s^* = \beta\alpha$ for Brock–Mirman; for richer models, on a trusted reference (time iteration, perturbation).
3. **Ergodic-set coverage.** Scatter test states in the $(K_t, \log z_t)$ plane; the cloud should trace the training ergodic set — not collapse to a point, not spread like a grid.
4. **Sanity checks.** Verify accounting identities (budget, market clearing, non-negativity); check that ergodic moments (mean K , $\text{std}(z)$, autocorrelations) match analytic or pre-computed values.

Green flags: low Euler error, matches benchmark, ergodic coverage, identities hold. **Red flags:** NaNs, fat tails, clamped policy, violated identities.

When no closed-form benchmark exists, draw inspiration from the **VVUQ literature** — Verification, Validation & Uncertainty Quantification (Roache 1998; Oberkampf & Roy 2010; ASME V&V 10/20) — which formalises residual convergence, solution verification, and error-bound reporting for computational models.

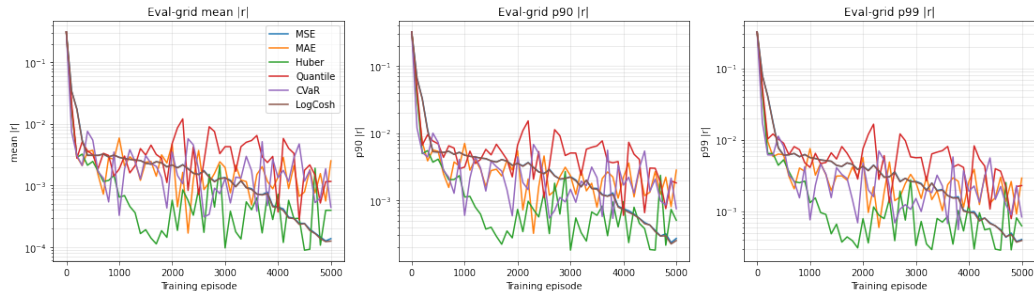
Choice of Training Loss: Six Kernels

The loss kernel is a **modeling decision**, not a default. Same Euler residual r , same network, same data, only the reduction $r \mapsto \ell$ changes:

Kernel	Formula	Character
MSE	$\frac{1}{B} \sum r^2$	quadratic; canonical default
MAE	$\frac{1}{B} \sum r $	constant gradient $\Rightarrow$ stalls at small $ r $
Huber	quad. for $ r \leq \delta$, linear above	robust hybrid, but kink at $ r = \delta$
Quantile	pinball at $\tau = 0.9$	trains the upper τ -quantile only
CVaR	mean of top 10% of $ r $	explicitly trains the worst-case states
LogCosh	$\frac{1}{B} \sum \log \cosh(r)$	C^∞ : $\frac{1}{2}r^2$ near 0, $ r - \log 2$ in tails, no kink

Same Brock–Mirman model, full depreciation so $s^* = \alpha\beta$ is in closed form; same network (2×32 swish, sigmoid head); same Adam optimizer; same CRN training stream. Six runs differ *only* in the reduction applied to the residual vector.

Loss Kernels: Convergence on Brock–Mirman



Mean / p_{90} / p_{99} of the absolute relative Euler error $|r|$ on a fixed evaluation grid, vs. training episode (log scale, 5000 episodes per kernel). **LogCosh** and **MSE** reach the lowest mean residual. **Huber** and **CVaR** produce the narrowest tail. **MAE** stalls at a noise floor.

Loss-Kernel Comparison: What to Take Away

- ▶ **LogCosh converges most smoothly**: the only C^∞ kernel that is MSE-like near $r = 0$ and MAE-like in the tails, no kink anywhere. Lowest-risk default when the residual distribution is unknown.
- ▶ **MAE stalls** at a noise floor: constant gradient magnitude prevents smaller steps as residuals shrink.
- ▶ **Huber is robust** but its kink at $|r|=\delta$ can introduce small oscillations.
- ▶ **Quantile and CVaR** narrow p_{99} at the cost of mean residual. Use them when worst-case policy quality matters (binding constraints, rare-but-consequential states, regulatory back-tests).
- ▶ **Evaluate on the economic metric** — Euler error along simulated paths, in CE-welfare-loss units — not on the training loss itself. The CE ranking is generally not the training-loss ranking.

Practical default

MSE remains the canonical default, but **LogCosh** is a smooth, kink-free alternative that should be benchmarked alongside it before defaulting; **Huber** / **Quantile** / **CVaR** when the worst-case state matters.

Companion notebook: `code/01_brock_mirman/01_05_StochasticBM_LossComparison.ipynb`.

Part IV

Parallelization & HPC

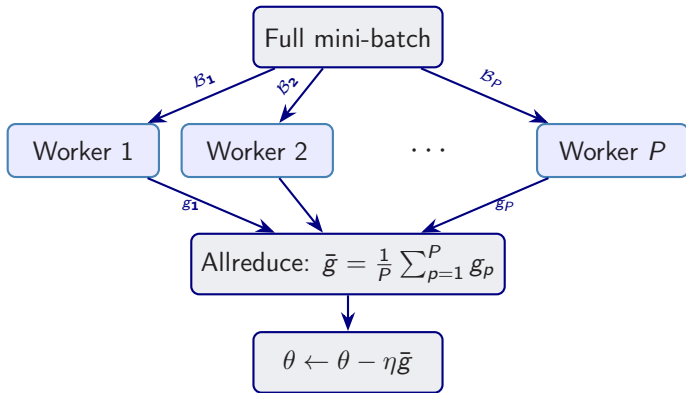
*“To pull a bigger wagon, it is easier to add more oxen
than to grow a gigantic ox.”*

— Gropp, Lusk & Skjellum (1999)

Using MPI: Portable Parallel Programming

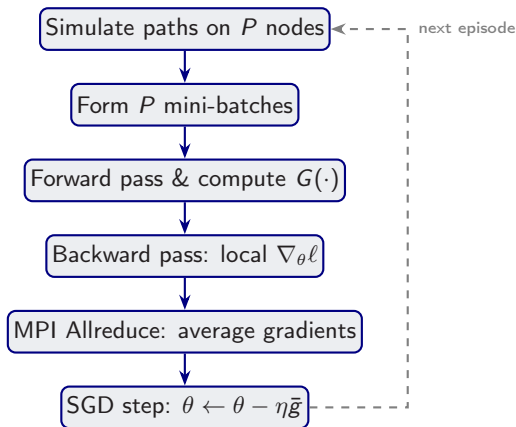
Data Parallelism for DEQNs

Idea: distribute training data across **multiple workers** (GPUs/CPU's).



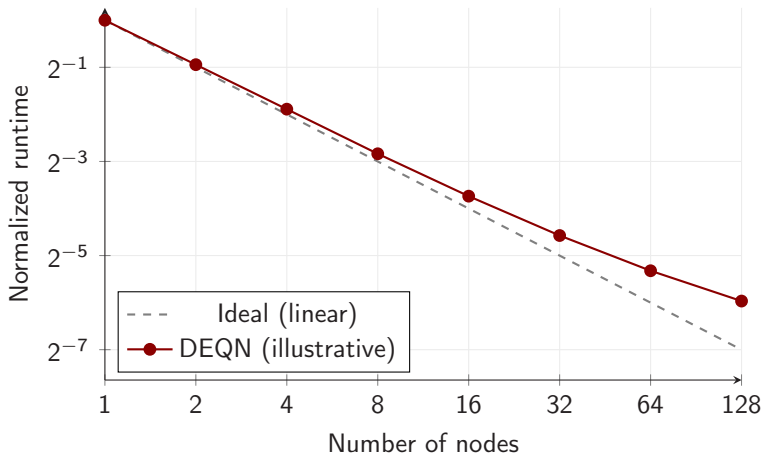
Implemented via **Horovod**/MPI. Each worker computes a local gradient; **allreduce** averages them.

DEQN Parallelization Scheme



Communication cost $\sim \mathcal{O}(|\theta|)$ per step, independent of data size.

Illustrative Scaling Pattern (Schematic)



Schematic message only: synchronous data parallelism is often close to linear for moderate worker counts; large clusters become increasingly communication-limited.

Preview: the Hands-on Exercises

Exercise notebooks 01_03/01_04 extend Brock–Mirman in two directions:

1. *Endogenous labor* with a non-negativity constraint on leisure, introducing a *Karush–Kuhn–Tucker* multiplier $\mu_t \geq 0$.
2. A *6-period OLG* economy with a borrowing constraint on the young, again KKT with $\mu_t \geq 0$.

Why this matters for DEQNs. A KKT system imposes complementary slackness $\mu \geq 0$, $g \geq 0$, $\mu g = 0$. This cannot be encoded as a smooth equation, so we replace it by the *Fischer–Burmeister* function

$$\phi_{\text{FB}}(a, b) = a + b - \sqrt{a^2 + b^2}, \quad \phi_{\text{FB}}(a, b) = 0 \iff a \geq 0, b \geq 0, ab = 0,$$

which is smooth everywhere except at the origin and is fully differentiable by autodiff.

Exercise loss. The notebooks sum a squared *Euler* residual and a squared *FB* residual for each constraint; both are minimized jointly. Details: see Session 3 (IRBC) and the KKT sub-section of the lecture script's IRBC chapter.

Hands-on Notebooks

Four notebooks accompany this lecture:

1. `01_01_Brock_Mirman_1972_DEQN.ipynb`
Deterministic Brock–Mirman, **exogenous uniform sampling** of states.
2. `01_02_Brock_Mirman_Uncertainty_DEQN.ipynb`
Stochastic Brock–Mirman ($\varrho > 0$, $\sigma_z > 0$), **simulation-based sampling** on the ergodic set.
3. `01_03_DEQN_Exercises_Blanks.ipynb` (exercises, blank template for student populate)
4. `01_04_DEQN_Exercises_Solutions.ipynb` (exercises, solved)

Notebooks `01_01`–`01_02` illustrate the sampling progression: uniform random $\rightarrow$ simulated trajectories. Notebooks `01_03`–`01_04` add KKT multipliers + Fischer–Burmeister + the 6-period OLG structure previewed on the previous slide.

All notebooks are in `day2/Scheidegger_Kubler/code/01_brock_mirman/`.

Questions?

Simon Scheidegger

HEC, University of Lausanne

<https://sischei.github.io>

Materials: [https:](https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models)

[//github.com/sischei/Deep_](https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models)

[Learning_for_Solving_And_](https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models)

[Estimating_Dynamic_Economic_Models](https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models)

Next up:

DEQN Exercises